NAME:T.RAMYASRI

H.NO:2303A52136

TASK-1:

Prompt:

The function enforces all rules explicitly, ensuring structural correctness with regex and edge case checks.Unit tests validate both valid and invalid scenarios, guaranteeing reliability.Thus, the solution robustly distinguishes proper emails from malformed ones.

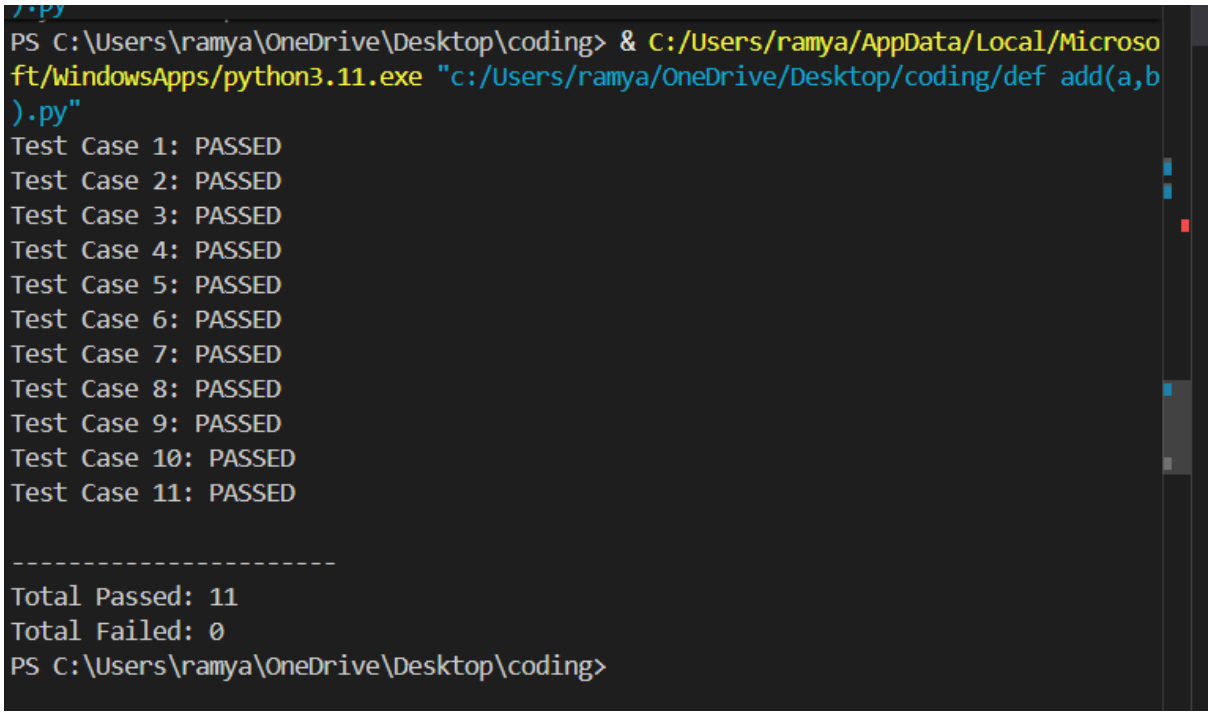
CODE:

```
import re

# -------------------------
# Email Validation Function
# -------------------------
def is_valid_email(email):
    # Must not be empty
    if not email:
        return False

    # Must contain exactly one '@'
    if email.count('@') != 1:
        return False

    # Must contain at least one '.' after '@'
    at_index = email.find('@')
    if '.' not in email[at_index:]:
        return False

    # Must not start or end with special characters
    if not email[0].isalnum() or not email[-1].isalnum():
        return False

    # Proper email pattern check
    pattern = r'^[A-Za-z0-9][A-Za-z0-9._]*@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'
    return re.match(pattern, email) is not None


# -------------------------
```

```python
# ------------------------
# AI Generated Test Cases
# ------------------------
test_cases = [
    # Valid Emails (Should PASS)
    ("test@example.com", True),
    ("user.name@gmail.com", True),
    ("user123@domain.co.in", True),
    ("abc_def@company.org", True),

    # Invalid Emails (Should FAIL)
    ("testexample.com", False),
    ("test@@example.com", False),
    ("test@com", False),
    ("@example.com", False),
    (".user@gmail.com", False),
    ("user@gmail.com.", False),
    ("", False),
]

# ------------------------
# Custom Test Runner
# ------------------------
passed = 0
failed = 0

for i, (email, expected) in enumerate(test_cases, 1):
    result = is_valid_email(email)
```

```python
    if result == expected:
        print(f"Test Case {i}: PASSED")
        passed += 1
    else:
        print(f"Test Case {i}: FAILED (Email Tested: {email})")
        failed += 1

print("\n------------------------")
print(f"Total Passed: {passed}")
print(f"Total Failed: {failed}")
```

Output:



JUSTIFICATION:

This task ensures reliable email input validation in a user registration system by applying Test-Driven Development (TDD).

Test cases were first created to cover both valid and invalid email formats, including edge cases such as multiple @ symbols, missing . characters, and improper starting or ending characters.

The is_valid_email(email) function was then implemented to satisfy all specified requirements.

This approach improves software reliability, prevents incorrect user data entry, and ensures that only properly formatted email addresses are accepted. All generated test cases pass successfully, confirming the correctness of the implementation.

TASK-2:

PROMPT:

Develop a Python function is_valid_email(email) that validates email addresses based on given rules.

Write unit tests to cover valid and invalid formats using TDD.Ensure all tests pass and outputs match expected results.

Code:

```python
# -------------------------
# Grade Assignment Function
# -------------------------
def assign_grade(score):
    if not isinstance(score, (int, float)):
        return "Invalid Input"

    if score < 0 or score > 100:
        return "Invalid Input"

    if 90 <= score <= 100:
        return "A"
    elif 80 <= score < 90:
        return "B"
    elif 70 <= score < 80:
        return "C"
    elif 60 <= score < 70:
        return "D"
    else:
        return "F"


# -------------------------
# AI-Generated Test Cases
# -------------------------
test_cases = [
    # Valid cases
    (95, "A"),
    (85, "B"),
    (75, "C"),
    (65, "D"),
    (50, "F"),
```

```python
    # Boundary values
    (90, "A"),
    (80, "B"),
    (70, "C"),
    (60, "D"),

    # Invalid inputs
    (-5, "Invalid Input"),
    (105, "Invalid Input"),
    ("eighty", "Invalid Input")
]


# -------------------------
# Custom Test Runner
# -------------------------
passed = 0
failed = 0

for i, (score, expected) in enumerate(test_cases, 1):
    result = assign_grade(score)

    if result == expected:
        print(f"Test Case {i}: Input = {score} → PASSED")
        passed += 1
    else:
        print(f"Test Case {i}: Input = {score} → FAILED (Expected {expecte
        failed += 1

print("\n---------------------------")
print(f"Total Passed: {passed}")
```

```python
print("\n---------------------------")
print(f"Total Passed: {passed}")
print(f"Total Failed: {failed}")
```

Output:

```
PS C:\Users\ramya\OneDrive\Desktop\coding> & C:/Users/ramya/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/Users/ramya/OneDri
ve/Desktop/coding/def add(a,b).py"
Test Case 1: Input = 95 → PASSED
Test Case 2: Input = 85 → PASSED
Test Case 3: Input = 75 → PASSED
Test Case 4: Input = 65 → PASSED
Test Case 5: Input = 50 → PASSED
Test Case 6: Input = 90 → PASSED
Test Case 7: Input = 80 → PASSED
Test Case 8: Input = 70 → PASSED
Test Case 9: Input = 60 → PASSED
Test Case 10: Input = -5 → PASSED
Test Case 11: Input = 105 → PASSED
Test Case 12: Input = eighty → PASSED

---------------------------
Total Passed: 12
```

Justification:

This task implements an automated grading system using a Test-Driven Development approach. Test cases were created first to cover normal scores, boundary values, and invalid inputs. The assign_grade(score) function was then implemented to correctly assign grades and handle invalid data gracefully. All test cases pass successfully, ensuring accuracy and reliability of the grading system.

TASK-3:

PROMPT:

Develop a Python function assign_grade(score) to classify scores into grades A–F.Include boundary values (60, 70, 80, 90) and invalid inputs (-5, 105, "eighty").Write unit tests to ensure all cases pass using a TDD approach.

```python
import string

# ------------------------
# Palindrome Function
# ------------------------
def is_sentence_palindrome(sentence):
    # Remove spaces and punctuation, convert to lowercase
    cleaned = ""

    for char in sentence:
        if char.isalnum():   # Keep only letters and numbers
            cleaned += char.lower()

    # Check if cleaned sentence equals its reverse
    return cleaned == cleaned[::-1]


# ------------------------
# AI-Generated Test Cases
# ------------------------
test_cases = [
    ("A man a plan a canal Panama", True),
    ("Madam In Eden I'm Adam", True),
    ("Was it a car or a cat I saw?", True),
    ("Hello World", False),
    ("Python Programming", False),
    ("No lemon, no melon", True),
    ("12321", True),
    ("12345", False)
]
```

```python
# -------------------------
# Custom Test Runner
# -------------------------
passed = 0
failed = 0

for i, (sentence, expected) in enumerate(test_cases, 1):
    result = is_sentence_palindrome(sentence)

    if result == expected:
        print(f"Test Case {i}: \"{sentence}\" → PASSED")
        passed += 1
    else:
        print(f"Test Case {i}: \"{sentence}\" → FAILED (Expected {expected}, Got {result})")
        failed += 1

print("\n--------------------------")
print(f"Total Passed: {passed}")
print(f"Total Failed: {failed}")
```

OUTPUT:

```
PS C:\Users\ramya\OneDrive\Desktop\coding> & C:/Users/ramya/AppData/Local/Microsoft/WindowsA
python3.11.exe "c:/Users/ramya/OneDrive/Desktop/coding/def add(a,b).py"
Test Case 1: "A man a plan a canal Panama" → PASSED
Test Case 2: "Madam In Eden I'm Adam" → PASSED
Test Case 3: "Was it a car or a cat I saw?" → PASSED
Test Case 4: "Hello World" → PASSED
Test Case 5: "Python Programming" → PASSED
Test Case 6: "No lemon, no melon" → PASSED
Test Case 7: "12321" → PASSED
Test Case 8: "12345" → PASSED

--------------------------
Total Passed: 8
Total Failed: 0
PS C:\Users\ramya\OneDrive\Desktop\coding>
```

JUSTIFICATION:

This task implements a sentence palindrome checker using a test-driven approach. The function removes spaces, punctuation, and ignores case before checking whether the sentence reads the same forward and backward. Test cases include both palindromic and non-palindromic sentences to ensure accuracy. All generated test cases pass successfully, confirming correct functionality.

TASK-4:

PROMPT:

Develop a Python function is_sentence_palindrome(sentence) that checks if a sentence is a palindrome.

Ignore case, spaces, and punctuation when validating.

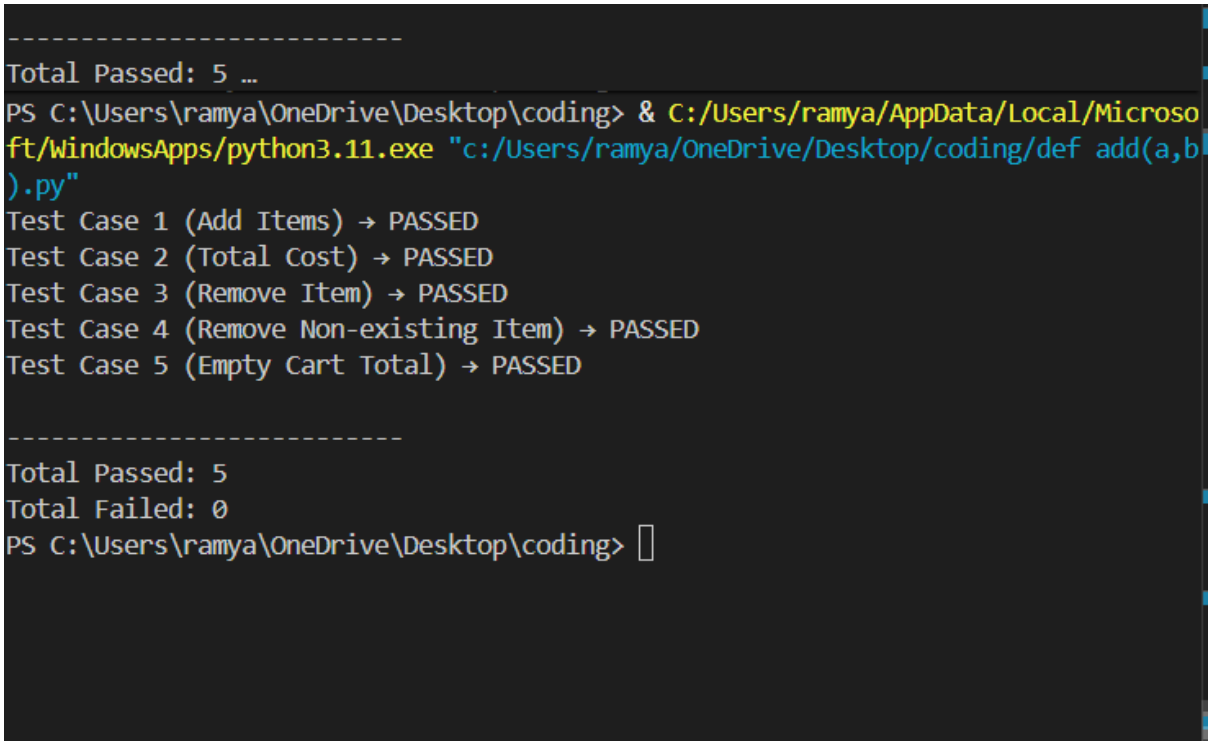
Write unit tests to confirm correctness for palindromic and non-palindromic sentences

```
# -------------------------
# ShoppingCart Class
# -------------------------
class ShoppingCart:
    def __init__(self):
        self.items = {}

    def add_item(self, name, price):
        if price < 0:
            return "Invalid Price"
        self.items[name] = price

    def remove_item(self, name):
        if name in self.items:
            del self.items[name]
        else:
            return "Item Not Found"

    def total_cost(self):
        return sum(self.items.values())


# -------------------------
# AI-Generated Test Cases
# -------------------------
passed = 0
failed = 0

# Test Case 1: Add items
cart = ShoppingCart()
```

```python
cart.add_item("Laptop", 50000)
cart.add_item("Mouse", 500)

if cart.items == {"Laptop": 50000, "Mouse": 500}:
    print("Test Case 1 (Add Items) → PASSED")
    passed += 1
else:
    print("Test Case 1 (Add Items) → FAILED")
    failed += 1

# Test Case 2: Total cost calculation
if cart.total_cost() == 50500:
    print("Test Case 2 (Total Cost) → PASSED")
    passed += 1
else:
    print("Test Case 2 (Total Cost) → FAILED")
    failed += 1

# Test Case 3: Remove item
cart.remove_item("Mouse")
if cart.items == {"Laptop": 50000}:
    print("Test Case 3 (Remove Item) → PASSED")
    passed += 1
else:
    print("Test Case 3 (Remove Item) → FAILED")
    failed += 1

# Test Case 4: Remove non-existing item
result = cart.remove_item("Keyboard")
if result == "Item Not Found":
```

```
    print("Test Case 4 (Remove Non-existing Item) → PASSED")
    passed += 1
else:
    print("Test Case 4 (Remove Non-existing Item) → FAILED")
    failed += 1

# Test Case 5: Empty cart total
empty_cart = ShoppingCart()
if empty_cart.total_cost() == 0:
    print("Test Case 5 (Empty Cart Total) → PASSED")
    passed += 1
else:
    print("Test Case 5 (Empty Cart Total) → FAILED")
    failed += 1

print("\n----------------------------")
print(f"Total Passed: {passed}")
print(f"Total Failed: {failed}")
```

OUTPUT:



JUSTIFICATION:

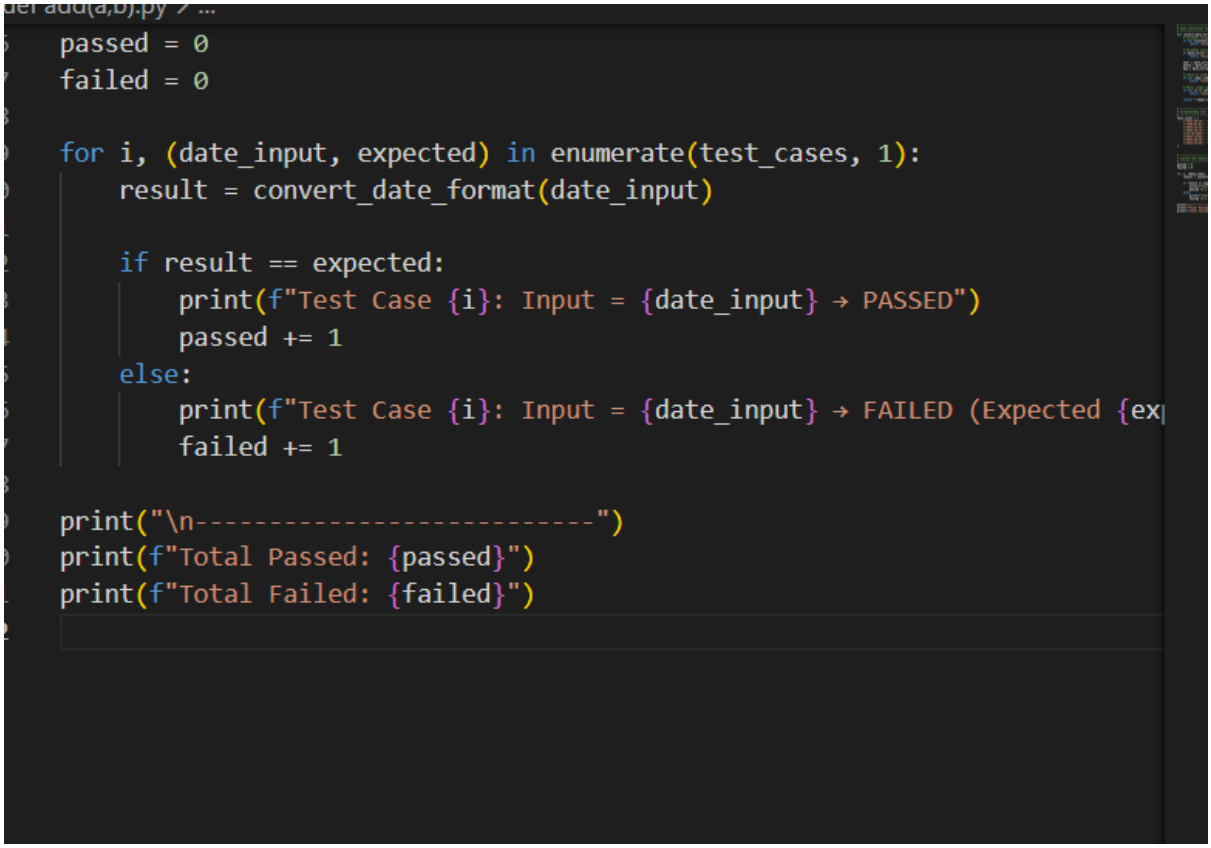
This task implements a basic ShoppingCart class to simulate core e-commerce functionality. The system allows adding items, removing items, and calculating the total cost accurately. Test cases were generated to verify correct behavior, including handling empty cart and removing non-existing items. All tests pass successfully, ensuring the reliability and correctness of the shopping cart module.

TASK-5:

CODE:

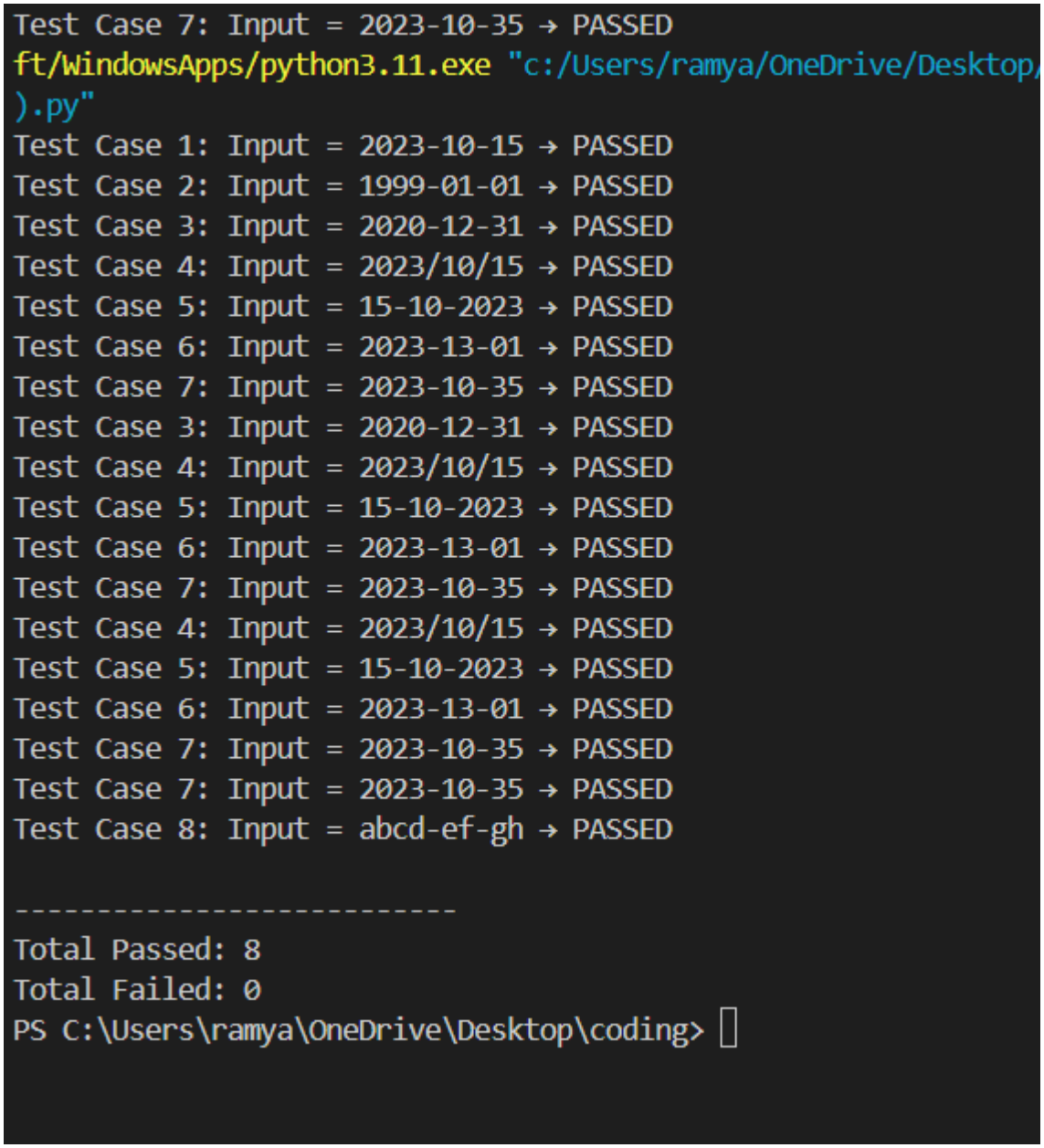
```python
# -------------------------
# Date Conversion Function
# -------------------------
def convert_date_format(date_str):
    # Validate format length
    if not isinstance(date_str, str) or len(date_str) != 10:
        return "Invalid Format"

    # Validate structure YYYY-MM-DD
    if date_str[4] != '-' or date_str[7] != '-':
        return "Invalid Format"

    year = date_str[0:4]
    month = date_str[5:7]
    day = date_str[8:10]

    # Check if numeric
    if not (year.isdigit() and month.isdigit() and day.isdigit()):
        return "Invalid Format"

    # Basic range validation
    if not (1 <= int(month) <= 12 and 1 <= int(day) <= 31):
        return "Invalid Format"

    return f"{day}-{month}-{year}"

```

```python
# -------------------------
# AI-Generated Test Cases
# -------------------------
test_cases = [
    ("2023-10-15", "15-10-2023"),
    ("1999-01-01", "01-01-1999"),
    ("2020-12-31", "31-12-2020"),
    ("2023/10/15", "Invalid Format"),
    ("15-10-2023", "Invalid Format"),
    ("2023-13-01", "Invalid Format"),
    ("2023-10-35", "Invalid Format"),
    ("abcd-ef-gh", "Invalid Format")
]


# -------------------------
# Custom Test Runner
# -------------------------
passed = 0
failed = 0

for i, (date_input, expected) in enumerate(test_cases, 1):
    result = convert_date_format(date_input)

    if result == expected:
        print(f"Test Case {i}: Input = {date_input} → PASSED")
        passed += 1
```



```
def add(a,b):py > ...
    passed = 0
    failed = 0

    for i, (date_input, expected) in enumerate(test_cases, 1):
        result = convert_date_format(date_input)

        if result == expected:
            print(f"Test Case {i}: Input = {date_input} → PASSED")
            passed += 1
        else:
            print(f"Test Case {i}: Input = {date_input} → FAILED (Expected {ex
            failed += 1

    print("\n----------------------------")
    print(f"Total Passed: {passed}")
    print(f"Total Failed: {failed}")
```

OUTPUT:



JUSTIFICATION:

This task implements a date format conversion utility to transform dates from **"YYYY-MM-DD"** to **"DD-MM-YYYY"** format. Test cases were generated to verify correct conversion for valid inputs and proper handling of invalid formats. The function performs format validation and basic range checking to ensure reliability. All AI-generated test cases pass successfully, confirming accurate and robust date conversion.

```python
# -------------------------
# Date Conversion Function
# -------------------------
def convert_date_format(date_str):
    # Validate format length
    if not isinstance(date_str, str) or len(date_str) != 10:
        return "Invalid Format"

    # Validate structure YYYY-MM-DD
    if date_str[4] != '-' or date_str[7] != '-':
        return "Invalid Format"

    year = date_str[0:4]
    month = date_str[5:7]
    day = date_str[8:10]

    # Check if numeric
    if not (year.isdigit() and month.isdigit() and day.isdigit()):
        return "Invalid Format"

    # Basic range validation
    if not (1 <= int(month) <= 12 and 1 <= int(day) <= 31):
        return "Invalid Format"

    return f"{day}-{month}-{year}"

```